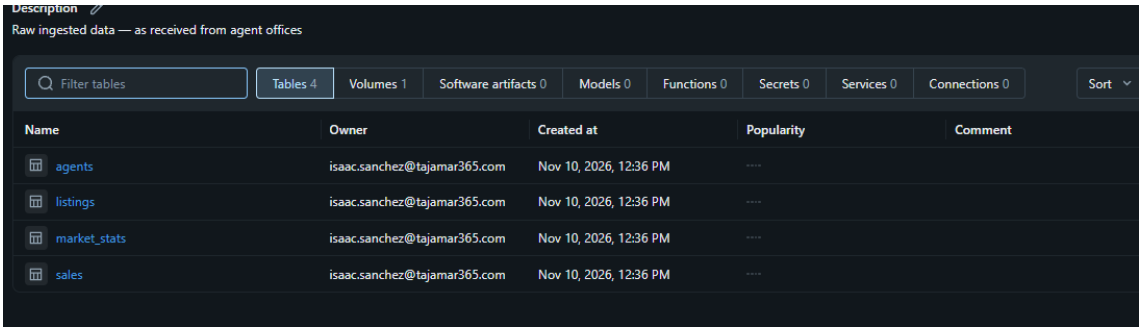
Contenido

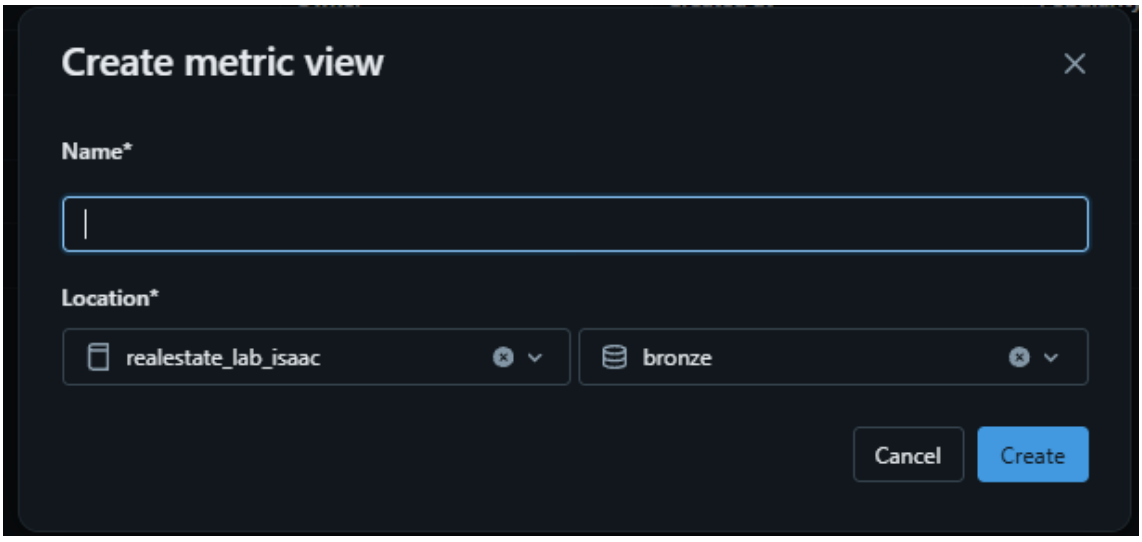
Tarea sin uso de portátil: Crear un perfil de datos en el Explorador de catálogos. .. 2
Ejercicios ... 3
Ejercicio 1 ... 3
Crear el catalofo ... 3
Insertar datos ... 4
Conversión a bronce ... 5
Ejercicio 2 ... 5
Ejercicio 2.1 ... 5
Ejercicio 2.2 ... 6
Ejercicio 3 ... 6
Ejercicio 3.1 ... 6
Ejercicio 3.3 ... 7
Ejercicio 4 ... 8
Ejercicio 4.1 ... 8
Ejercicio 4.2 ... 8
Ejercicio 4.3 ... 9
Ejercicio 4.4 ... 9
Ejercicio 5 ... 10
Ejercicio 5.1 ... 10
Ejercicio 5.2 ... 10
Ejercicio 5.3 ... 10
Ejercicio 6 ... 11
Ejercicio 6.2 ... 11
Limpieza ... 12

Tarea sin uso de portátil: Crear un perfil de datos en el Explorador de catálogos.

1. Abra **el Explorador de catálogos** desde el panel de navegación izquierdo.
2. Navegue al catálogo **de realestate_lab** → esquema **bronce → tabla de listados** .
3. Seleccione la pestaña **Calidad y, a continuación, seleccione Habilitar** para abrir el cuadro de diálogo **de Supervisión de la calidad de los datos** .



4. En la tarjeta **Perfilado de datos** , seleccione **Configurar** para abrir el cuadro de diálogo **Crear monitor** .



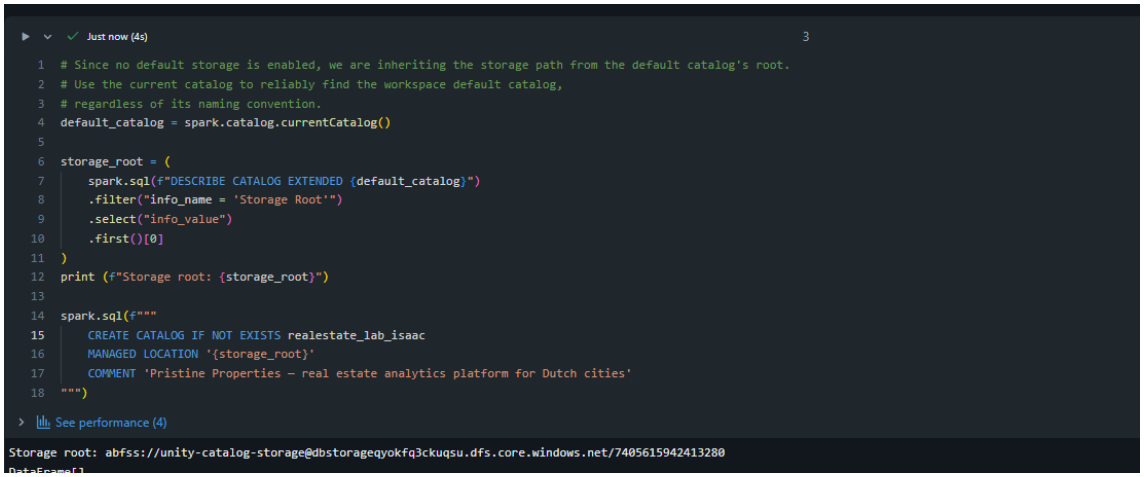
5. En **Perfilado** → **Tipo de perfil** , seleccione **Instantánea** ; esto es adecuado para una tabla de uso general como los listados.
6. Seleccione **Crear** para crear el monitor y generar el primer perfil.
7. Una vez creado el monitor, abra el **panel de control** generado (seleccione **Ver panel de control** en la pestaña Calidad) y seleccione **Actualizar** para

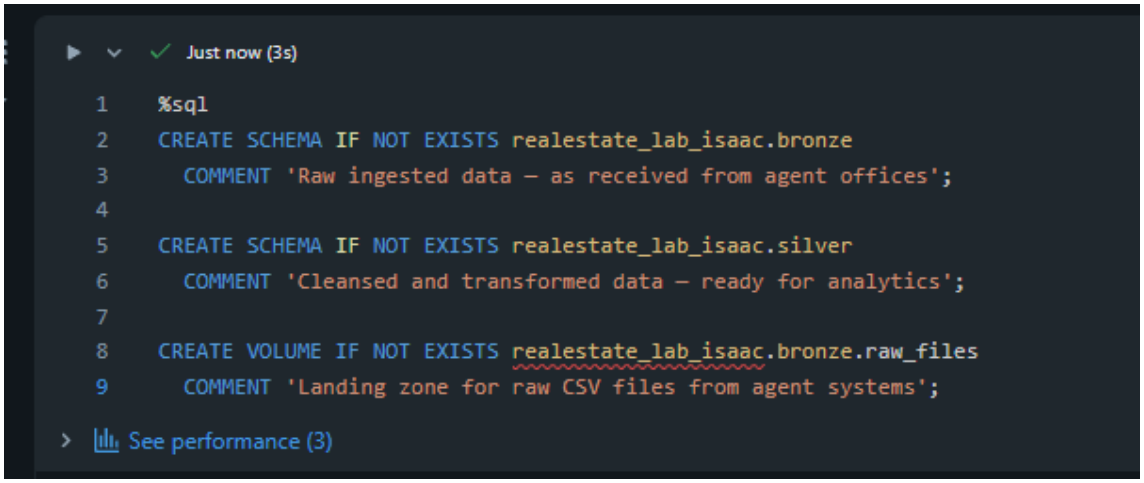
rellenarlo. Si posteriormente necesita regenerar las métricas, vuelva a la pestaña Calidad y seleccione **Actualizar y refrescar** .

Ejercicios

Ejercicio 1

Crear el catalogo





Insertar datos

```
agents_csv = """agent_id,agent_name,agency,city,phone
101,Sophie van den Berg,Makelaardij De Kust,Amsterdam,+31-20-1234567
102,Jan de Vries,Vastgoed Noord,Rotterdam,+31-10-9876543
103,Maria Hendriks,De Hoek Makelaars,Utrecht,+31-30-4567890
104,Thomas Bakker,Haagse Woning,The Hague,+31-70-2345678
105,Lisa Smits,Brainport Vastgoed,Eindhoven,+31-40-3456789
106,Pieter Jansen,Stadspark Makelaars,Groningen,+31-50-8765432
107,Emma Koster,Markdal Vastgoed,Breda,+31-76-5678901
108,Robert Mulder,Canal Ring Realty,Amsterdam,+31-20-9012345"""

sales_csv = """sale_id,property_id,sale_price,sale_date,days_on_market
1,PROP-005,315000.00,2026-03-15,38
2,PROP-011,192000.00,2026-04-20,43
3,PROP-016,172500.00,2026-04-30,35"""

market_csv = """city,Jan,Feb,Mar,Apr,May,Jun,Jul,Aug,Sep,Oct,Nov,Dec
Amsterdam,385000,392000,401000,415000,422000,438000,445000,441000,428000,418000,405000,398000
Rotterdam,295000,298000,305000,312000,318000,325000,321000,315000,308000,301000,295000,290000
Utrecht,310000,315000,322000,328000,335000,342000,348000,344000,336000,328000,318000,312000
The Hague,275000,280000,288000,295000,302000,308000,312000,306000,298000,291000,283000,278000
Eindhoven,258000,262000,268000,275000,282000,288000,293000,289000,281000,273000,265000,260000
Groningen,195000,198000,203000,208000,213000,218000,222000,219000,214000,208000,201000,196000
Breda,245000,249000,255000,262000,268000,274000,279000,275000,268000,260000,253000,247000"""

base = "/Volumes/realestate_lab_isaac/bronze/raw_files"
dbutils.fs.put(f"{base}/property_listings.csv",   listings_csv, overwrite=True)
dbutils.fs.put(f"{base}/real_estate_agents.csv",  agents_csv,   overwrite=True)
dbutils.fs.put(f"{base}/property_sales.csv",      sales_csv,    overwrite=True)
dbutils.fs.put(f"{base}/market_stats_wide.csv",   market_csv,   overwrite=True)
print("CSV files written to volume.")
```

> See performance (4)

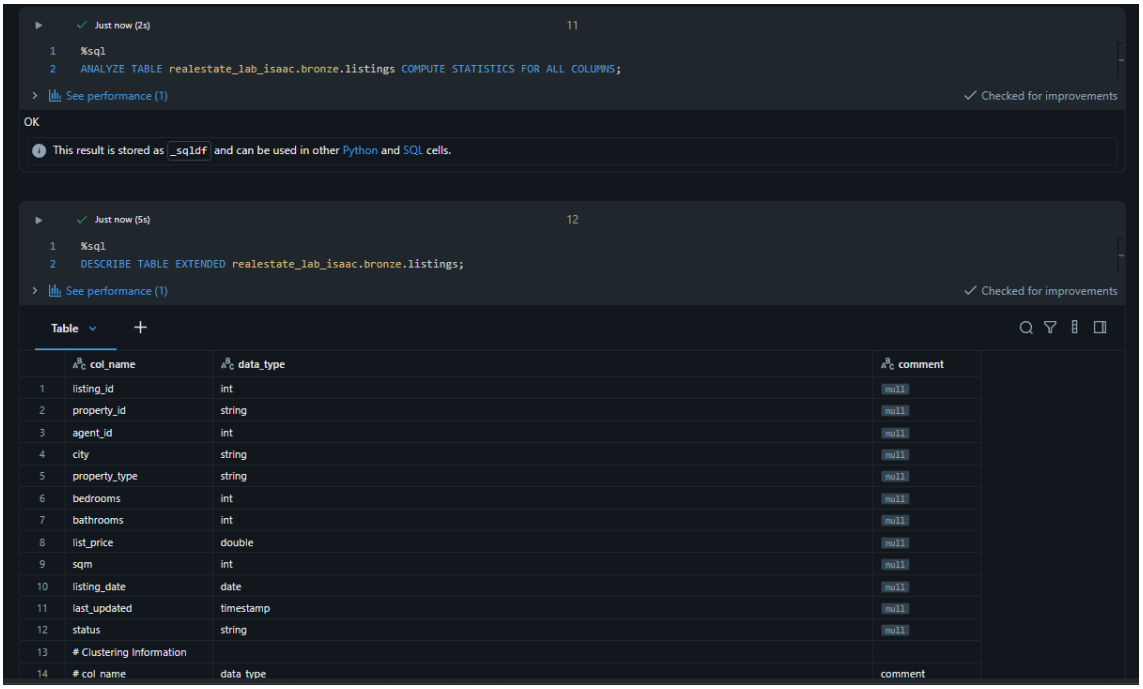
```
Wrote 2990 bytes.
Wrote 517 bytes.
Wrote 160 bytes.
Wrote 704 bytes.
CSV files written to volume.
```

Conversión a bronce

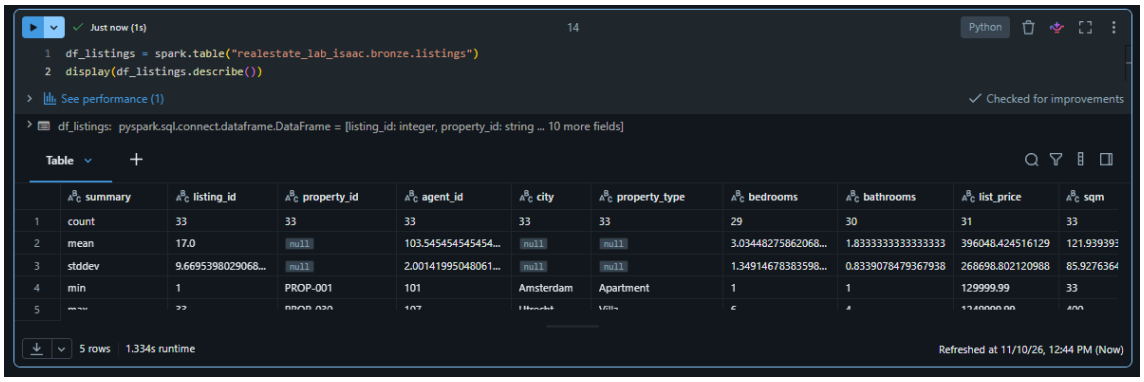


Ejercicio 2

Ejercicio 2.1



Ejercicio 2.2



```
df_listings = spark.table("realestate_lab_isaac.bronze.listings")
display(df_listings.describe())
```

df_listings: pyspark.sql.connect.dataframe.DataFrame = [listing_id: integer, property_id: string ... 10 more fields]

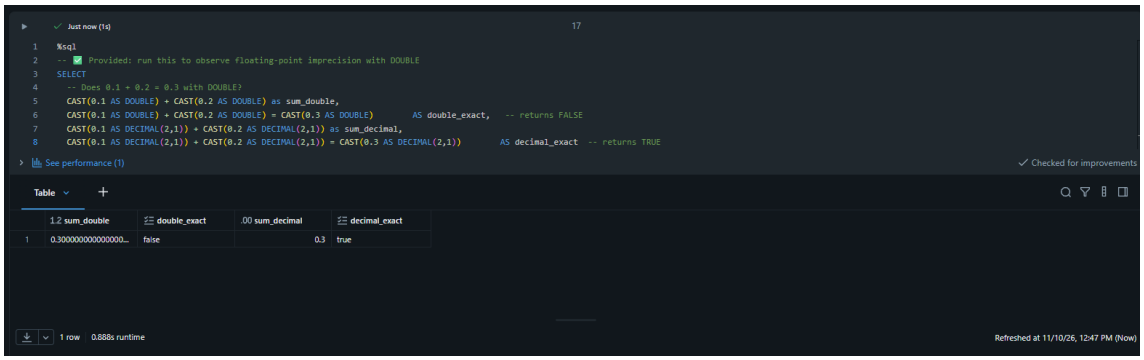
	summary	listing_id	property_id	agent_id	city	property_type	bedrooms	bathrooms	list_price	sqm
1	count	33	33	33	33	33	29	30	31	33
2	mean	17.0	null	103.545454545454...	null	null	3.03448275862068...	1.8333333333333333	396048.424516129	121.939393
3	stddev	9.6695398029068...	null	2.00141995048061...	null	null	1.34914678383598...	0.8339078479367938	268698.802120988	85.9276364
4	min	1	PROP-001	101	Amsterdam	Apartment	1	1	129999.99	33
5	max	33	PROP-030	107	Utrecht	Villa	6	4	1249999.99	400

5 rows | 1.334s runtime

Refreshed at 11/10/26, 12:44 PM (Now)

Ejercicio 3

Ejercicio 3.1

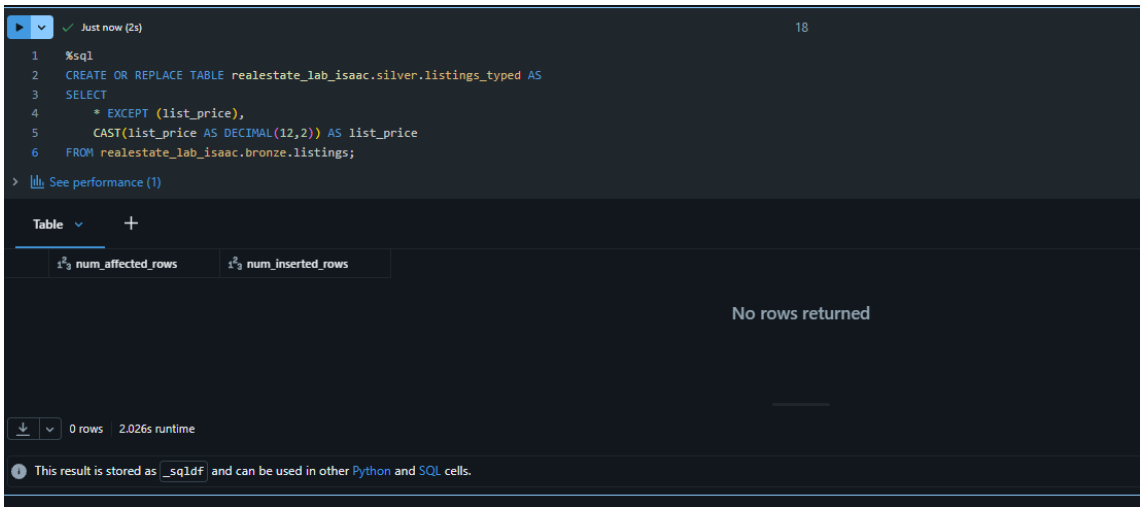


```
%sql
-- ✅ Provided: run this to observe floating-point imprecision with DOUBLE
SELECT
  -- Does 0.1 + 0.2 = 0.3 with DOUBLE?
  CAST(0.1 AS DOUBLE) + CAST(0.2 AS DOUBLE) as sum_double,
  CAST(0.1 AS DOUBLE) + CAST(0.2 AS DOUBLE) = CAST(0.3 AS DOUBLE)       AS double_exact,   -- returns FALSE
  CAST(0.1 AS DECIMAL(2,1)) + CAST(0.2 AS DECIMAL(2,1)) as sum_decimal,
  CAST(0.1 AS DECIMAL(2,1)) + CAST(0.2 AS DECIMAL(2,1)) = CAST(0.3 AS DECIMAL(2,1))       AS decimal_exact  -- returns TRUE
```

	sum_double	double_exact	sum_decimal	decimal_exact
1	0.30000000000000...	false	0.3	true

1 row | 0.888s runtime

Refreshed at 11/10/26, 12:47 PM (Now)



```
%sql
CREATE OR REPLACE TABLE realestate_lab_isaac.silver.listings_typed AS
SELECT
    * EXCEPT (list_price),
    CAST(list_price AS DECIMAL(12,2)) AS list_price
FROM realestate_lab_isaac.bronze.listings;
```

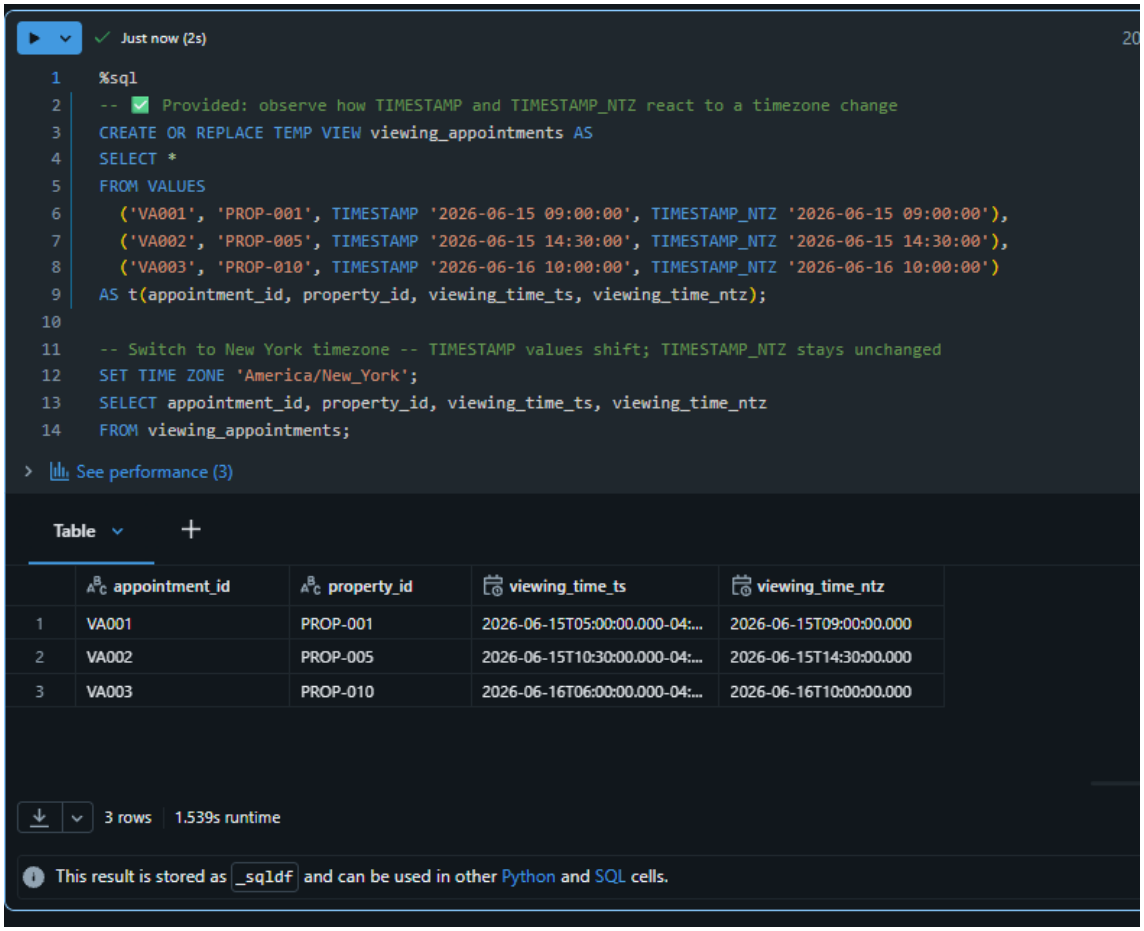
num_affected_rows	num_inserted_rows

No rows returned

0 rows | 2.026s runtime

This result is stored as `_sqldf` and can be used in other Python and SQL cells.

Ejercicio 3.2



```
%sql
-- ✅ Provided: observe how TIMESTAMP and TIMESTAMP_NTZ react to a timezone change
CREATE OR REPLACE TEMP VIEW viewing_appointments AS
SELECT *
FROM VALUES
  ('VA001', 'PROP-001', TIMESTAMP '2026-06-15 09:00:00', TIMESTAMP_NTZ '2026-06-15 09:00:00'),
  ('VA002', 'PROP-005', TIMESTAMP '2026-06-15 14:30:00', TIMESTAMP_NTZ '2026-06-15 14:30:00'),
  ('VA003', 'PROP-010', TIMESTAMP '2026-06-16 10:00:00', TIMESTAMP_NTZ '2026-06-16 10:00:00')
AS t(appointment_id, property_id, viewing_time_ts, viewing_time_ntz);

-- Switch to New York timezone -- TIMESTAMP values shift; TIMESTAMP_NTZ stays unchanged
SET TIME ZONE 'America/New_York';
SELECT appointment_id, property_id, viewing_time_ts, viewing_time_ntz
FROM viewing_appointments;
```

Just now (2s)

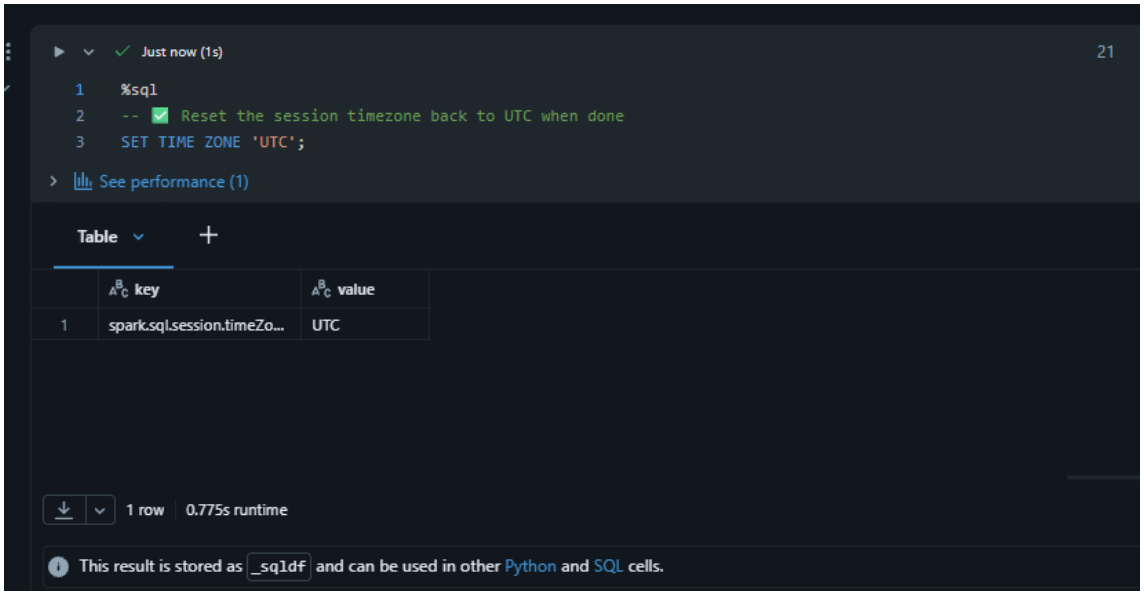
See performance (3)

Table

	appointment_id	property_id	viewing_time_ts	viewing_time_ntz
1	VA001	PROP-001	2026-06-15T05:00:00.000-04:...	2026-06-15T09:00:00.000
2	VA002	PROP-005	2026-06-15T10:30:00.000-04:...	2026-06-15T14:30:00.000
3	VA003	PROP-010	2026-06-16T06:00:00.000-04:...	2026-06-16T10:00:00.000

3 rows | 1.539s runtime

This result is stored as `_sqldf` and can be used in other Python and SQL cells.



```
%sql
-- ✅ Reset the session timezone back to UTC when done
SET TIME ZONE 'UTC';
```

Just now (1s)

See performance (1)

Table

	key	value
1	spark.sql.session.timeZo...	UTC

1 row | 0.775s runtime

This result is stored as `_sqldf` and can be used in other Python and SQL cells.

Ejercicio 3.3



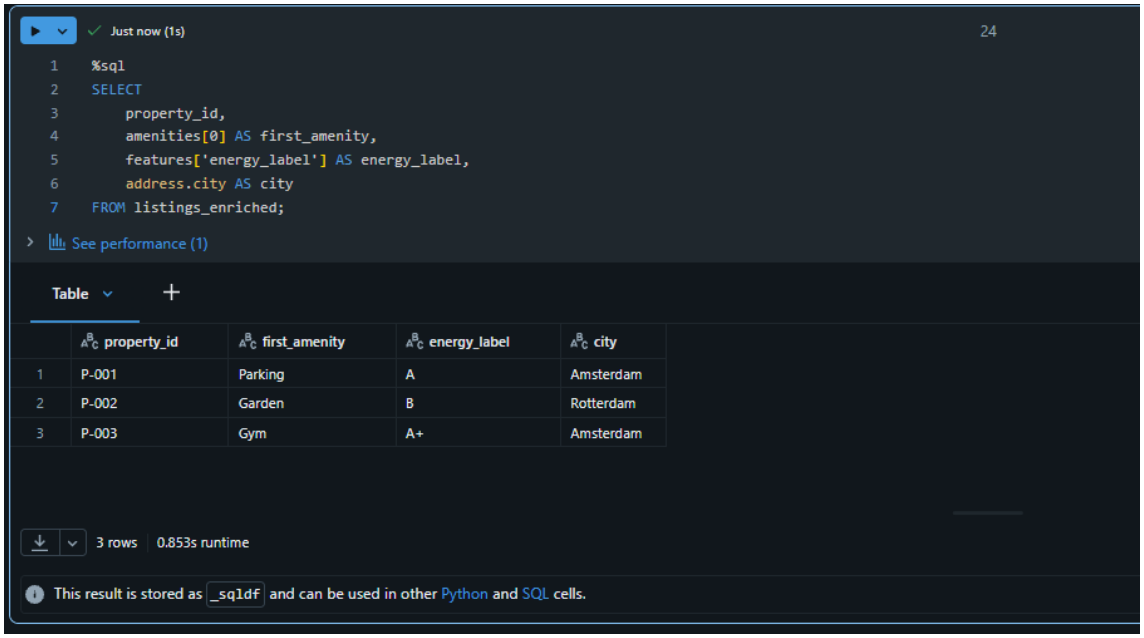
```
%sql
CREATE OR REPLACE TEMP VIEW listings_enriched AS
SELECT * FROM VALUES
    ('P-001', ARRAY('Parking', 'Balcony', 'Storage'), MAP('energy_label', 'A', 'heating', 'central'), NAMED_STRUCT('street', 'Keizersgracht 123', 'city', 'Amsterdam', 'postal_code', '1015 CJ')),
    ('P-002', ARRAY('Garden', 'Pool'), MAP('energy_label', 'B', 'heating', 'electric'), NAMED_STRUCT('street', 'Coolsingel 45', 'city', 'Rotterdam', 'postal_code', '3012 AA')),
    ('P-003', ARRAY('Gym', 'Security'), MAP('energy_label', 'A+', 'heating', 'district'), NAMED_STRUCT('street', 'Zuidas 88', 'city', 'Amsterdam', 'postal_code', '1082 ME'))
AS v(property_id, amenities, features, address);
```

Just now (<1s)

See performance (1)

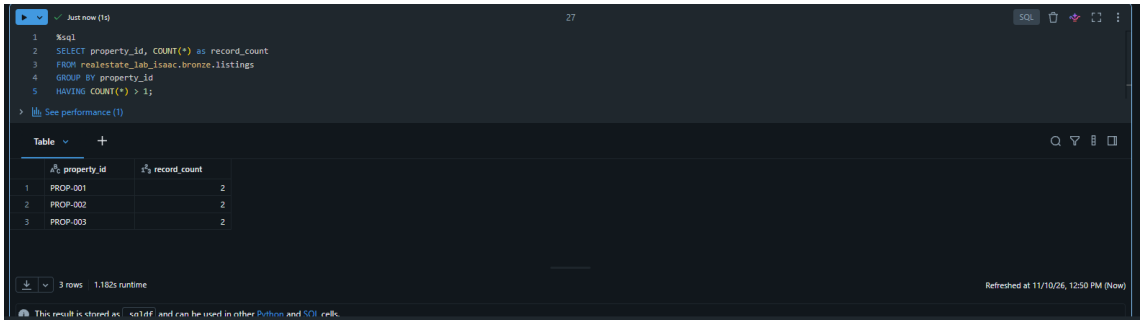
Checked for improvements

OK

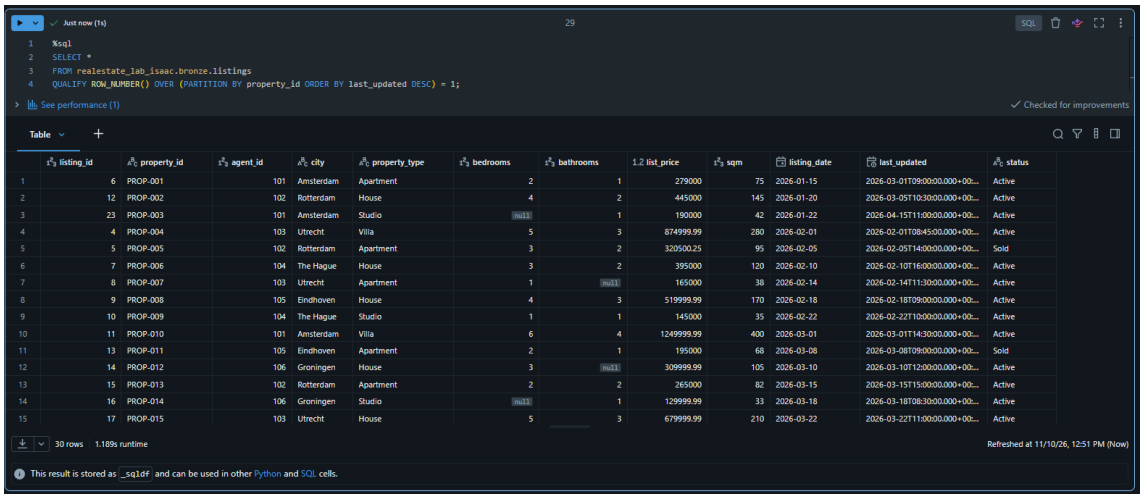


Ejercicio 4

Ejercicio 4.1



Ejercicio 4.2



```
%sql
SELECT *
FROM realestate_lab_isaac.bronze.listings
QUALIFY ROW_NUMBER() OVER (PARTITION BY property_id ORDER BY last_updated DESC) = 1;
```

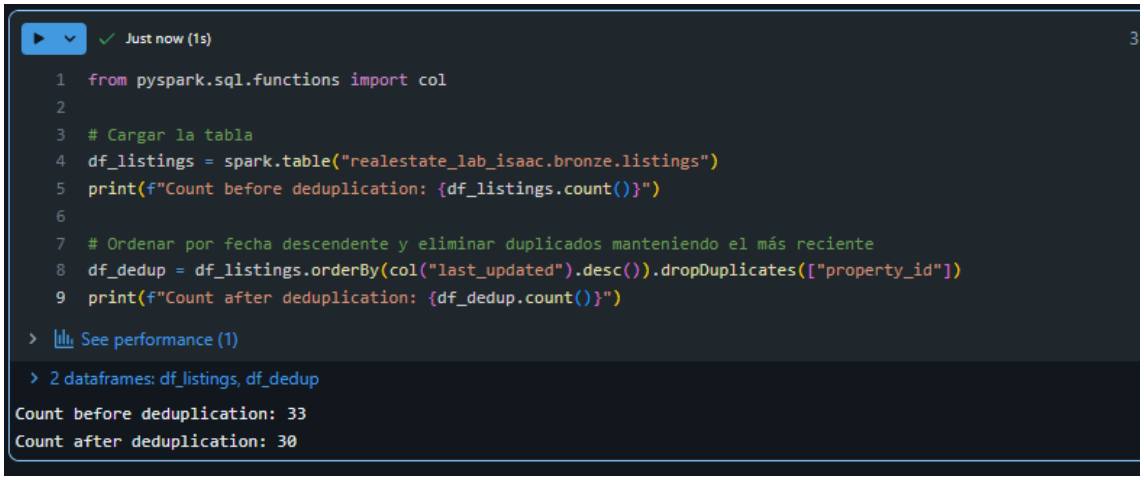
	listing_id	property_id	agent_id	city	property_type	bedrooms	bathrooms	list_price	sqm	listing_date	last_updated	status
1	6	PROP-001	101	Amsterdam	Apartment	2	1	279000	75	2026-01-15	2026-03-01T09:00:00.000+00:...	Active
2	12	PROP-002	102	Rotterdam	House	4	2	445000	145	2026-01-20	2026-03-05T10:30:00.000+00:...	Active
3	23	PROP-003	101	Amsterdam	Studio	null	1	190000	42	2026-01-22	2026-04-15T11:00:00.000+00:...	Active
4	4	PROP-004	103	Utrecht	Villa	5	3	874999.99	280	2026-02-01	2026-02-01T08:45:00.000+00:...	Active
5	5	PROP-005	102	Rotterdam	Apartment	3	2	320500.25	95	2026-02-05	2026-02-05T14:00:00.000+00:...	Sold
6	7	PROP-006	104	The Hague	House	3	2	395000	120	2026-02-10	2026-02-10T16:00:00.000+00:...	Active
7	8	PROP-007	103	Utrecht	Apartment	1	null	165000	38	2026-02-14	2026-02-14T11:30:00.000+00:...	Active
8	9	PROP-008	105	Eindhoven	House	4	3	519999.99	170	2026-02-18	2026-02-18T09:00:00.000+00:...	Active
9	10	PROP-009	104	The Hague	Studio	1	1	145000	35	2026-02-22	2026-02-22T10:00:00.000+00:...	Active
10	11	PROP-010	101	Amsterdam	Villa	6	4	1249999.99	400	2026-03-01	2026-03-01T14:30:00.000+00:...	Active
11	13	PROP-011	105	Eindhoven	Apartment	2	1	195000	68	2026-03-08	2026-03-08T09:00:00.000+00:...	Sold
12	14	PROP-012	106	Groningen	House	3	null	309999.99	105	2026-03-10	2026-03-10T12:00:00.000+00:...	Active
13	15	PROP-013	102	Rotterdam	Apartment	2	2	265000	82	2026-03-15	2026-03-15T15:00:00.000+00:...	Active
14	16	PROP-014	106	Groningen	Studio	null	1	129999.99	33	2026-03-18	2026-03-18T08:30:00.000+00:...	Active
15	17	PROP-015	103	Utrecht	House	5	3	679999.99	210	2026-03-22	2026-03-22T11:00:00.000+00:...	Active

30 rows 1.189s runtime

Refreshed at 11/10/26, 12:51 PM (Now)

This result is stored as `_sqldf` and can be used in other Python and SQL cells.

Ejercicio 4.3



```
from pyspark.sql.functions import col

# Cargar la tabla
df_listings = spark.table("realestate_lab_isaac.bronze.listings")
print(f"Count before deduplication: {df_listings.count()}")

# Ordenar por fecha descendente y eliminar duplicados manteniendo el más reciente
df_dedup = df_listings.orderBy(col("last_updated").desc()).dropDuplicates(["property_id"])
print(f"Count after deduplication: {df_dedup.count()}")
```

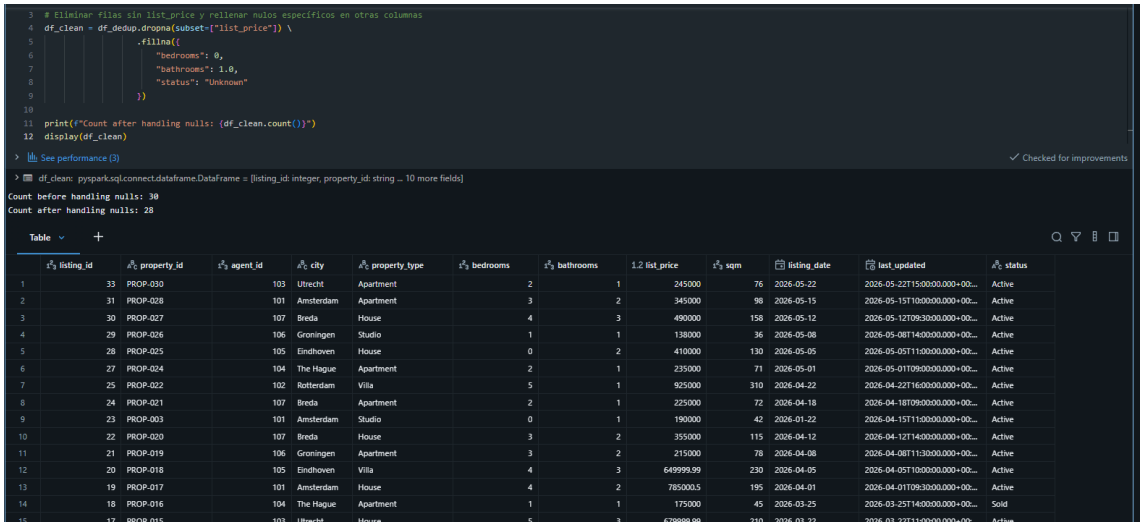
Just now (1s)

See performance (1)

2 dataframes: df_listings, df_dedup

```
Count before deduplication: 33
Count after deduplication: 30
```

Ejercicio 4.4



```
# Eliminar filas sin list_price y rellenar nulos específicos en otras columnas
df_clean = df_dedup.dropna(subset=["list_price"]) \
                   .fillna({
                       "bedrooms": 0,
                       "bathrooms": 1.0,
                       "status": "Unknown"
                   })

print(f"Count after handling nulls: {df_clean.count()}")
display(df_clean)
```

See performance (3)

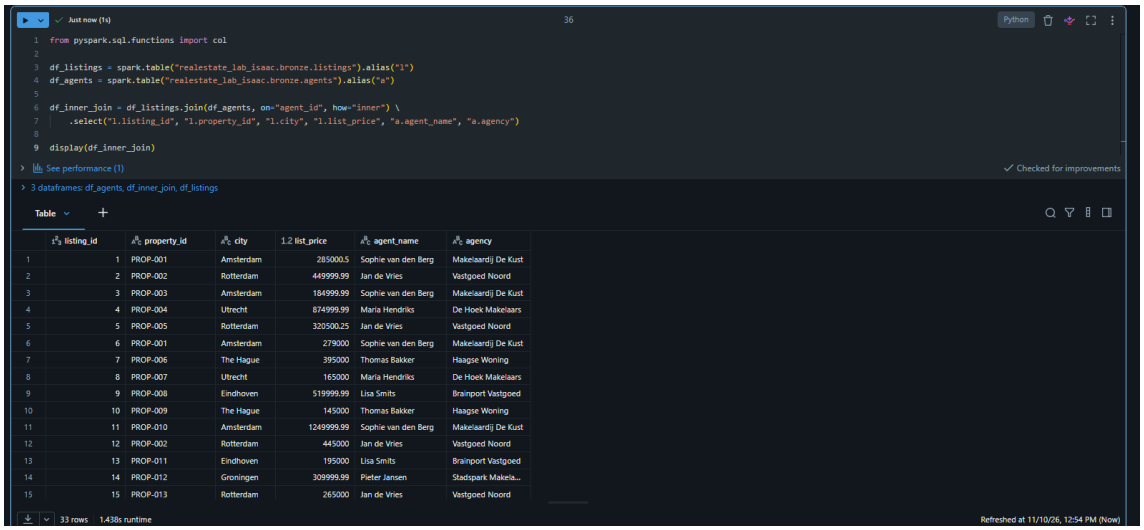
df_clean: pyspark.sql.connect.dataframe.DataFrame = [listing_id: integer, property_id: string ... 10 more fields]

```
Count before handling nulls: 30
Count after handling nulls: 28
```

	listing_id	property_id	agent_id	city	property_type	bedrooms	bathrooms	list_price	sqm	listing_date	last_updated	status
1	33	PROP-030	103	Utrecht	Apartment	2	1	245000	76	2026-05-22	2026-05-22T15:00:00.000+00:...	Active
2	31	PROP-028	101	Amsterdam	Apartment	3	2	345000	98	2026-05-15	2026-05-15T10:00:00.000+00:...	Active
3	30	PROP-027	107	Breda	House	4	3	490000	158	2026-05-12	2026-05-12T09:30:00.000+00:...	Active
4	29	PROP-026	106	Groningen	Studio	1	1	138000	36	2026-05-08	2026-05-08T14:00:00.000+00:...	Active
5	28	PROP-025	105	Eindhoven	House	0	2	410000	130	2026-05-05	2026-05-05T11:00:00.000+00:...	Active
6	27	PROP-024	104	The Hague	Apartment	2	1	235000	71	2026-05-01	2026-05-01T09:00:00.000+00:...	Active
7	25	PROP-022	102	Rotterdam	Villa	5	1	925000	310	2026-04-22	2026-04-22T16:00:00.000+00:...	Active
8	24	PROP-021	107	Breda	Apartment	2	1	225000	72	2026-04-18	2026-04-18T09:00:00.000+00:...	Active
9	23	PROP-003	101	Amsterdam	Studio	0	1	190000	42	2026-01-22	2026-04-15T11:00:00.000+00:...	Active
10	22	PROP-020	107	Breda	House	3	2	355000	115	2026-04-12	2026-04-12T14:00:00.000+00:...	Active
11	21	PROP-019	106	Groningen	Apartment	3	2	215000	78	2026-04-08	2026-04-08T11:30:00.000+00:...	Active
12	20	PROP-018	105	Eindhoven	Villa	4	3	649999.99	230	2026-04-05	2026-04-05T10:00:00.000+00:...	Active
13	19	PROP-017	101	Amsterdam	House	4	2	785000.5	195	2026-04-01	2026-04-01T09:30:00.000+00:...	Active
14	18	PROP-016	104	The Hague	Apartment	1	1	175000	45	2026-03-25	2026-03-25T14:00:00.000+00:...	Sold

Ejercicio 5

Ejercicio 5.1



```
from pyspark.sql.functions import col

df_listings = spark.table("realestate_lab_isaac.bronze.listings").alias("l")
df_agents = spark.table("realestate_lab_isaac.bronze.agents").alias("a")

df_inner_join = df_listings.join(df_agents, on="agent_id", how="inner") \
    .select("l.listing_id", "l.property_id", "l.city", "l.list_price", "a.agent_name", "a.agency")

display(df_inner_join)
```

	listing_id	property_id	city	list_price	agent_name	agency
1	1	PROP-001	Amsterdam	285000.5	Sophie van den Berg	Makelaardij De Kust
2	2	PROP-002	Rotterdam	449999.99	Jan de Vries	Vastgoed Noord
3	3	PROP-003	Amsterdam	184999.99	Sophie van den Berg	Makelaardij De Kust
4	4	PROP-004	Utrecht	874999.99	Maria Hendriks	De Hoek Makelaars
5	5	PROP-005	Rotterdam	320500.25	Jan de Vries	Vastgoed Noord
6	6	PROP-001	Amsterdam	279000	Sophie van den Berg	Makelaardij De Kust
7	7	PROP-006	The Hague	395000	Thomas Bakker	Haagse Woning
8	8	PROP-007	Utrecht	165000	Maria Hendriks	De Hoek Makelaars
9	9	PROP-008	Eindhoven	519999.99	Lisa Smits	Brainport Vastgoed
10	10	PROP-009	The Hague	145000	Thomas Bakker	Haagse Woning
11	11	PROP-010	Amsterdam	1249999.99	Sophie van den Berg	Makelaardij De Kust
12	12	PROP-002	Rotterdam	445000	Jan de Vries	Vastgoed Noord
13	13	PROP-011	Eindhoven	195000	Lisa Smits	Brainport Vastgoed
14	14	PROP-012	Groningen	309999.99	Pieter Jansen	Stadspark Makela...
15	15	PROP-013	Rotterdam	265000	Jan de Vries	Vastgoed Noord

33 rows · 1.438s runtime

Ejercicio 5.2



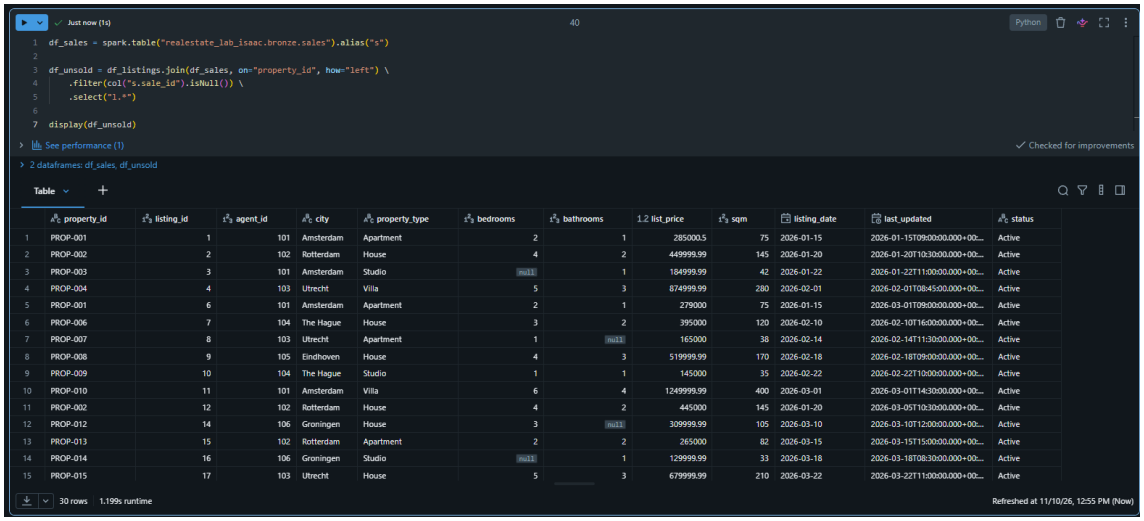
```
df_agents_no_listings = df_agents.join(df_listings, on="agent_id", how="left") \
    .filter(col("l.listing_id").isNull()) \
    .select("a.*")

display(df_agents_no_listings)
```

	agent_id	agent_name	agency	city	phone
1	108	Robert Mulder	Canal Ring Realty	Amsterdam	+31-20-90123...

1 row · 1.106s runtime

Ejercicio 5.3



```
df_sales = spark.table("realestate_lab_isaac.bronze.sales").alias("s")

df_unsold = df_listings.join(df_sales, on="property_id", how="left") \
    .filter(col("s.sale_id").isNull()) \
    .select("l.*")

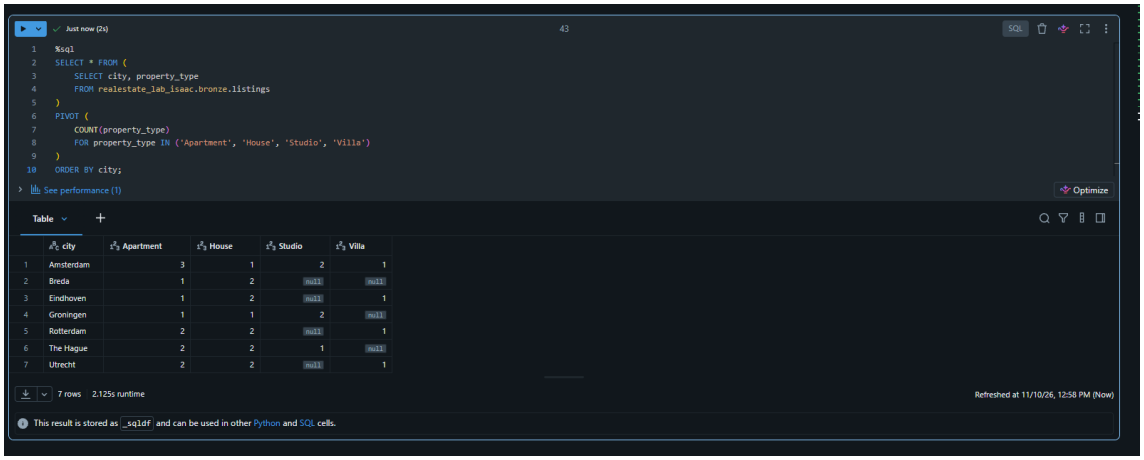
display(df_unsold)
```

	property_id	listing_id	agent_id	city	property_type	bedrooms	bathrooms	list_price	sqm	listing_date	last_updated	status
1	PROP-001	1	101	Amsterdam	Apartment	2	1	285000.5	75	2026-01-15	2026-01-15T09:00:00.000+00:...	Active
2	PROP-002	2	102	Rotterdam	House	4	2	449999.99	145	2026-01-20	2026-01-20T10:30:00.000+00:...	Active
3	PROP-003	3	101	Amsterdam	Studio	null	1	184999.99	42	2026-01-22	2026-01-22T11:00:00.000+00:...	Active
4	PROP-004	4	103	Utrecht	Villa	5	3	874999.99	280	2026-02-01	2026-02-01T08:45:00.000+00:...	Active
5	PROP-001	6	101	Amsterdam	Apartment	2	1	279000	75	2026-01-15	2026-03-01T09:00:00.000+00:...	Active
6	PROP-006	7	104	The Hague	House	3	2	395000	120	2026-02-10	2026-02-10T16:00:00.000+00:...	Active
7	PROP-007	8	103	Utrecht	Apartment	1	null	165000	38	2026-02-14	2026-02-14T11:30:00.000+00:...	Active
8	PROP-008	9	105	Eindhoven	House	4	3	519999.99	170	2026-02-18	2026-02-18T09:00:00.000+00:...	Active
9	PROP-009	10	104	The Hague	Studio	1	1	145000	35	2026-02-22	2026-02-22T10:00:00.000+00:...	Active
10	PROP-010	11	101	Amsterdam	Villa	6	4	1249999.99	400	2026-03-01	2026-03-01T14:30:00.000+00:...	Active
11	PROP-002	12	102	Rotterdam	House	4	2	445000	145	2026-01-20	2026-03-05T10:30:00.000+00:...	Active
12	PROP-012	14	106	Groningen	House	3	null	309999.99	105	2026-03-10	2026-03-10T12:00:00.000+00:...	Active
13	PROP-013	15	102	Rotterdam	Apartment	2	2	265000	82	2026-03-15	2026-03-15T15:00:00.000+00:...	Active
14	PROP-014	16	106	Groningen	Studio	null	1	129999.99	33	2026-03-18	2026-03-18T08:30:00.000+00:...	Active
15	PROP-015	17	103	Utrecht	House	5	3	679999.99	210	2026-03-22	2026-03-22T11:00:00.000+00:...	Active

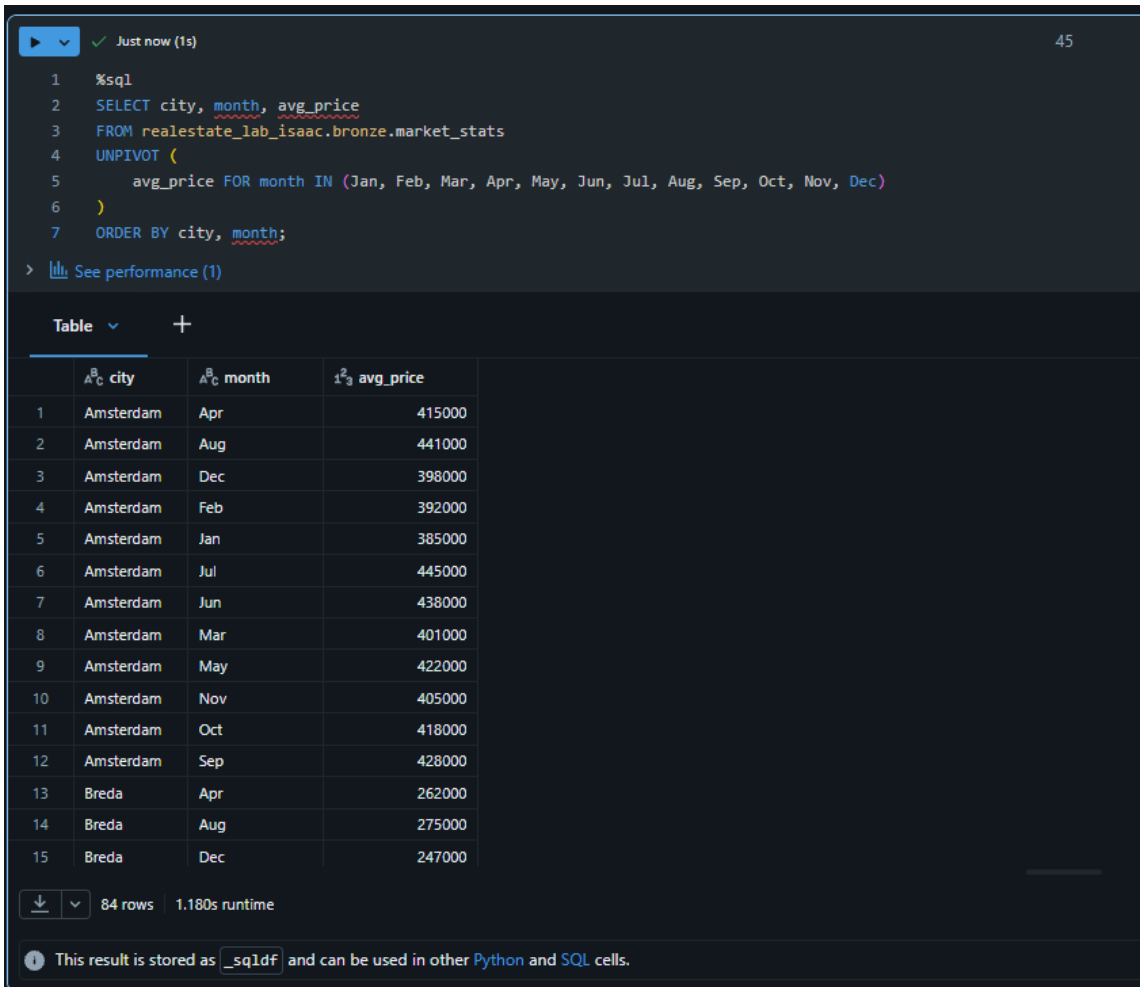
30 rows · 1.199s runtime

Ejercicio 6

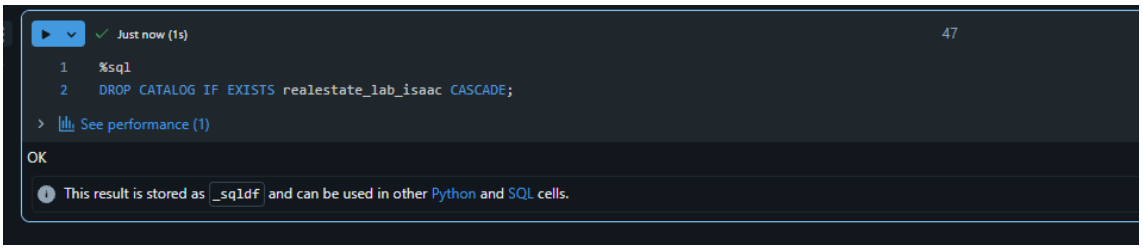
Ejercicio 6.1



Ejercicio 6.2



Limpieza


Just now (1s)
47
1 %sql
2 DROP CATALOG IF EXISTS realestate_lab_isaac CASCADE;
See performance (1)
OK
This result is stored as _sqldf and can be used in other Python and SQL cells.